互斥与死锁

Posted on 2023-08-26 11:39:09 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2UVOC0CihIdfguQFmv220vs5hAG)

简介

本文介绍如何使用互斥量保证共享数据的安全，并讲述死锁的相关处理方案。

锁的使用

我们可以通过mutex对共享数据进行夹所，防止多线程访问共享区造成数据不一致问题。如下，我们初始化一个共享变量 `shared_data` ，然后定义了一个互斥量 `std::mutex` ，接下来启动了两个线程，分别执行 `use_lock` 增加数据，和一个lambda表达式减少数据。
结果可以看到两个线程对于共享数据的访问是独占的，单位时间片只有一个线程访问并输出日志。

```
std::mutex  mtx1;
int shared_data = 100;

void use_lock() {
    while (true) {
        mtx1.lock();
        shared_data++;
        std::cout << "current thread is " << std::this_thread::get_id() << std::endl;
        std::cout << "sharad data is " << shared_data << std::endl;
        mtx1.unlock();
        std::this_thread::sleep_for(std::chrono::microseconds(10));
    }

}

void test_lock() {
    std::thread t1(use_lock);

    std::thread t2([]() {
        while (true) {
            mtx1.lock();
            shared_data--;
            std::cout << "current thread is " << std::this_thread::get_id() << std::endl;
            std::cout << "sharad data is " << shared_data << std::endl;
            mtx1.unlock();
            std::this_thread::sleep_for(std::chrono::microseconds(10));
        }
        });

    t1.join();
    t2.join();
}
```

lock_guard的使用

当然我们可以用 `lock_guard` 自动加锁和解锁,比如上面的函数可以等价简化为

```
1. void use_lock() {
2.     while (true) {
3.         std::lock_guard<std::mutex> lock(mtx1);
4.         shared_data++;
5.         std::cout << "current thread is " << std::this_thread::get_id() << std::endl;
6.         std::cout << "sharad data is " << shared_data << std::endl;
7.         std::this_thread::sleep_for(std::chrono::microseconds(10));
8.     }
9.
10. }
```

`lock_guard` 在作用域结束时自动调用其析构函数解锁，这么做的一个好处是简化了一些特殊情况从函数中返回的写法，比如异常或者条件不满足时，函数内部直接return，锁也会自动解开。

如何保证数据安全

有时候我们可以将对共享数据的访问和修改聚合到一个函数，在函数内加锁保证数据的安全性。但是对于读取类型的操作，即使读取函数是线程安全的，但是返回值抛给外边使用，存在不安全性。比如一个栈对象，我们要保证其在多线程访问的时候是安全的，可以在判断栈是否为空，判断操作内部我们可以加锁，但是判断结束后返回值就不在加锁了，就会存在线程安全问题。

比如我定义了如下栈, 对于多线程访问时判断栈是否为空，此后两个线程同时出栈，可能会造成崩溃。

```
template<typename T>
class threadsafe_stack1
{
private:
    std::stack<T> data;
    mutable std::mutex m;
public:
    threadsafe_stack1() {}
    threadsafe_stack1(const threadsafe_stack1& other)
    {
        std::lock_guard<std::mutex> lock(other.m);
        //①在构造函数的函数体（constructor body）内进行复制操作
        data = other.data;
    }
    threadsafe_stack1& operator=(const threadsafe_stack1&) = delete;
    void push(T new_value)
    {
        std::lock_guard<std::mutex> lock(m);
        data.push(std::move(new_value));
    }

    //问题代码
    T pop()
    {
        std::lock_guard<std::mutex> lock(m);
        auto element = data.top();
        data.pop();
        return element;
    }

    bool empty() const
    {
        std::lock_guard<std::mutex> lock(m);
        return data.empty();
    }
};
```

如下, 线程1和线程2先后判断栈都不为空，之后执行出战操作，会造成崩溃。

```
1. void test_threadsafe_stack1() {
2.     threadsafe_stack1<int> safe_stack;
3.     safe_stack.push(1);
4.
5.     std::thread t1([&safe_stack]() {
6.         if (!safe_stack.empty()) {
7.             std::this_thread::sleep_for(std::chrono::seconds(1));
8.             safe_stack.pop();
9.             }
10.        });
11.
12.    std::thread t2([&safe_stack]() {
13.        if (!safe_stack.empty()) {
14.            std::this_thread::sleep_for(std::chrono::seconds(1));
15.            safe_stack.pop();
16.        }
17.    });
18.
19.    t1.join();
20.    t2.join();
21. }
```

解决这个问题我们可以用抛出异常的方式，比如定义一个空栈的异常

```
1. struct empty_stack : std::exception
2. {
3.     const char* what() const throw();
4. };
```

然后实现我们的出栈函数

```
1. T pop()
2. {
3.     std::lock_guard<std::mutex> lock(m);
4.     if (data.empty()) throw empty_stack();
5.     auto element = data.top();
6.     data.pop();
7.     return element;
8. }
```

这么做就需要在外层使用的时候捕获异常。这是C++ 并发编程中提及的建议。但是我觉得可

以在函数pop内部再次判断栈是否为空，若为空则返回一个非法数据，这样比抛出异常好一些。但是如果T是一个复杂类型，我们很难定义一个非法值给外界知晓，这一点可以通过智能指针进行优化。之后我们再介绍更优化的方案，因为现在这个 `pop` 函数仍存在问题，比如 `T` 是一个 `vector<int>` 类型，那么在 `pop` 函数内部 `element` 就是 `vector<int>` 类型，开始 `element` 存储了一些int值，程序没问题，函数执行pop操作， 假设此时程序内存暴增，导致当程序使用的内存足够大时，可用的有效空间不够， 函数返回 `element` 时，就会就会存在 `vector` 做拷贝赋值时造成失败。即使我们捕获异常，释放部分空间但也会导致栈元素已经出栈，数据丢失了。这其实是内存管理不当造成的，但是C++ 并发编程一书中给出了优化方案。

```
1. struct empty_stack : std::exception
2. {
3.     const char* what() const throw();
4. };
5.
6. template<typename T>
7. class threadsafe_stack
8. {
9. private:
10.     std::stack<T> data;
11.     mutable std::mutex m;
12. public:
13.     threadsafe_stack() {}
14.     threadsafe_stack(const threadsafe_stack& other)
15.     {
16.         std::lock_guard<std::mutex> lock(other.m);
17.         //①在构造函数的函数体（constructor body）内进行复制操作
18.         data = other.data;
19.     }
20.     threadsafe_stack& operator=(const threadsafe_stack&) = delete;
21.     void push(T new_value)
22.     {
23.         std::lock_guard<std::mutex> lock(m);
24.         data.push(std::move(new_value));
25.     }
26.     std::shared_ptr<T> pop()
27.     {
28.         std::lock_guard<std::mutex> lock(m);
29.         //②试图弹出前检查是否为空栈
30.         if (data.empty()) throw empty_stack();
31.         //③改动栈容器前设置返回值
32.             std::shared_ptr<T> const res(std::make_shared<T>(data.top()));
33.             data.pop();
34.         return res;
35.     }
36.     void pop(T& value)
37.     {
38.         std::lock_guard<std::mutex> lock(m);
39.         if (data.empty()) throw empty_stack();
40.         value = data.top();
41.         data.pop();
42.     }
```

```
43.     bool empty() const
44.     {
45.         std::lock_guard<std::mutex> lock(m);
46.         return data.empty();
47.     }
48. };
```

我们提供了两个版本的 `pop` 操作，一个是带引用类型的参数的，一个是直接 `pop` 出智能指针类型，这样在 `pop` 函数内部减少了数据的拷贝，防止内存溢出，其实这两种做法确实是相比之前直接 `pop` 固定类型的值更节省内存，运行效率也好很多。我们也完全可以基于之前的思想，在 `pop` 时如果队列为空则返回空指针，这样比抛出异常更有好一些

```
1. std::shared_ptr<T> pop()
2. {
3.     std::lock_guard<std::mutex> lock(m);
4.     //②试图弹出前检查是否为空栈
5.     if (data.empty()) return nullptr;
6.     //③改动栈容器前设置返回值
7.     std::shared_ptr<T> const res(std::make_shared<T>(data.top()));
8.     data.pop();
9.     return res;
10. }
```

死锁怎么造成的

死锁一般是由于调运顺序不一致导致的，比如两个线程循环调用。当线程1先加锁A，再加锁B，而线程2先加锁B，再加锁A。那么在某一时刻就可能造成死锁。比如线程1对A已经加锁，线程2对B已经加锁，那么他们都希望彼此占有对方的锁，又不释放自己占有的锁导致了死锁。
举个例子

```
std::mutex  t_lock1;
std::mutex  t_lock2;
int m_1 = 0;
int m_2 = 1;

void dead_lock1() {
    while (true) {
        std::cout << "dead_lock1 begin " << std::endl;
        t_lock1.lock();
        m_1 = 1024;
        t_lock2.lock();
        m_2 = 2048;
        t_lock2.unlock();
        t_lock1.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(5));
        std::cout << "dead_lock2 end " << std::endl;
    }
}

void dead_lock2() {
    while (true) {
        std::cout << "dead_lock2 begin " << std::endl;
        t_lock2.lock();
        m_2 = 2048;
        t_lock1.lock();
        m_1 = 1024;
        t_lock1.unlock();
        t_lock2.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(5));
        std::cout << "dead_lock2 end " << std::endl;
    }
}
```

然后我们启动两个线程

```
void test_dead_lock() {
    std::thread t1(dead_lock1);
    std::thread t2(dead_lock2);
    t1.join();
    t2.join();
}
```

这样运行之后在某一个时刻一定会导致死锁。
实际工作中避免死锁的一个方式就是将加锁和解锁的功能封装为独立的函数，
这样能保证在独立的函数里执行完操作后就解锁，不会导致一个函数里使用多个锁的情况

```
1. //加锁和解锁作为原子操作解耦合，各自只管理自己的功能
2. void atomic_lock1() {
3.     std::cout << "lock1 begin lock" << std::endl;
4.     t_lock1.lock();
5.     m_1 = 1024;
6.     t_lock1.unlock();
7.     std::cout << "lock1 end lock" << std::endl;
8. }
9.
10. void atomic_lock2() {
11.     std::cout << "lock2 begin lock" << std::endl;
12.     t_lock2.lock();
13.     m_2 = 2048;
14.     t_lock2.unlock();
15.     std::cout << "lock2 end lock" << std::endl;
16. }
17.
18. void safe_lock1() {
19.     while (true) {
20.         atomic_lock1();
21.         atomic_lock2();
22.         std::this_thread::sleep_for(std::chrono::milliseconds(5));
23.     }
24. }
25.
26. void safe_lock2() {
27.     while (true) {
28.         atomic_lock2();
29.         atomic_lock1();
30.         std::this_thread::sleep_for(std::chrono::milliseconds(5));
31.     }
32. }
33.
34. void test_safe_lock() {
35.     std::thread t1(safe_lock1);
36.     std::thread t2(safe_lock2);
37.     t1.join();
38.     t2.join();
39. }
```

同时加锁

当我们无法避免在一个函数内部使用两个互斥量，并且都要解锁的情况，那我们可以采取同时加锁的方式。我们先定义一个类,假设这个类不推荐拷贝构造，但我们也提供了这个类的拷贝构造和移动构造

```cpp
class som_big_object {
public:
    som_big_object(int data) :_data(data) {}
    //拷贝构造
    som_big_object(const som_big_object& b2) :_data(b2._data) {
        _data = b2._data;
    }
    //移动构造
    som_big_object(som_big_object&& b2) :_data(std::move(b2._data)) {

    }
    //重载输出运算符
    friend std::ostream& operator << (std::ostream& os, const som_big_object& big_obj) {
        os << big_obj._data;
        return os;
    }

    //重载赋值运算符
    som_big_object& operator = (const som_big_object& b2) {
        if (this == &b2) {
            return *this;
        }
        _data = b2._data;
        return *this;
    }

    //交换数据
    friend void swap(som_big_object& b1, som_big_object& b2) {
        som_big_object temp = std::move(b1);
        b1 = std::move(b2);
        b2 = std::move(temp);
    }
private:
    int _data;
};
```

接下来我们定义一个类对上面的类做管理，为防止多线程情况下数据混乱， 包含了一个互斥量。

```
class big_object_mgr {
public:
    big_object_mgr(int data = 0) :_obj(data) {}
    void printinfo() {
        std::cout << "current obj data is " << _obj << std::endl;
    }
    friend void danger_swap(big_object_mgr& objm1, big_object_mgr& objm2);
    friend void safe_swap(big_object_mgr& objm1, big_object_mgr& objm2);
    friend void safe_swap_scope(big_object_mgr& objm1, big_object_mgr& objm2);
private:
    std::mutex _mtx;
    som_big_object _obj;
};
```

为了方便演示哪些交换是安全的，哪些是危险的，所以写了三个函数。

```
void danger_swap(big_object_mgr& objm1, big_object_mgr& objm2) {
    std::cout << "thread [ " << std::this_thread::get_id() << " ] begin" << std::endl;
    if (&objm1 == &objm2) {
        return;
    }
    std::lock_guard <std::mutex> gurad1(objm1._mtx);
    //此处为了故意制造死锁，我们让线程小睡一会
    std::this_thread::sleep_for(std::chrono::seconds(1));
    std::lock_guard<std::mutex> guard2(objm2._mtx);
    swap(objm1._obj, objm2._obj);
    std::cout << "thread [ " << std::this_thread::get_id() << " ] end" << std::endl;
}
```

`danger_swap` 是危险的交换方式。比如如下调用

```
void  test_danger_swap() {
    big_object_mgr objm1(5);
    big_object_mgr objm2(100);

    std::thread t1(danger_swap, std::ref(objm1), std::ref(objm2));
    std::thread t2(danger_swap, std::ref(objm2), std::ref(objm1));
    t1.join();
    t2.join();

    objm1.printinfo();
    objm2.printinfo();
}
```

这种调用方式存在隐患，因为 `danger_swap` 函数在两个线程中使用会造成互相竞争加锁的情况。
那就需要用锁同时锁住两个锁。

```
void safe_swap(big_object_mgr& objm1, big_object_mgr& objm2) {
    std::cout << "thread [ " << std::this_thread::get_id() << " ] begin" << std::endl;
    if (&objm1 == &objm2) {
        return;
    }

    std::lock(objm1._mtx, objm2._mtx);
    //领养锁管理它自动释放
    std::lock_guard <std::mutex> gurad1(objm1._mtx, std::adopt_lock);

    //此处为了故意制造死锁，我们让线程小睡一会
    std::this_thread::sleep_for(std::chrono::seconds(1));

    std::lock_guard <std::mutex> gurad2(objm2._mtx, std::adopt_lock);

    swap(objm1._obj, objm2._obj);
    std::cout << "thread [ " << std::this_thread::get_id() << " ] end" << std::endl;
}
```

比如下面的调用就是合理的

```
void test_safe_swap() {
    big_object_mgr objm1(5);
    big_object_mgr objm2(100);

    std::thread t1(safe_swap, std::ref(objm1), std::ref(objm2));
    std::thread t2(safe_swap, std::ref(objm2), std::ref(objm1));
    t1.join();
    t2.join();

    objm1.printinfo();
    objm2.printinfo();
}
```

当然上面加锁的方式可以简化，C++17 `scope_lock` 可以对多个互斥量同时加锁，并且自动释放

```
//上述代码可以简化为以下方式
void safe_swap_scope(big_object_mgr& objm1, big_object_mgr& objm2) {
    std::cout << "thread [ " << std::this_thread::get_id() << " ] begin" << std::endl;
    if (&objm1 == &objm2) {
        return;
    }

    std::scoped_lock  guard(objm1._mtx, objm2._mtx);
    //等价于
    //std::scoped_lock<std::mutex, std::mutex> guard(objm1._mtx, objm2._mtx);
    swap(objm1._obj, objm2._obj);
    std::cout << "thread [ " << std::this_thread::get_id() << " ] end" << std::endl;
}
```

层级锁

现实开发中常常很难规避同一个函数内部加多个锁的情况，我们要尽可能避免循环加锁，所以可以自定义一个层级锁，保证实际项目中对多个互斥量加锁时是有序的。

```
1. //层级锁
2. class hierarchical_mutex {
3. public:
4.     explicit hierarchical_mutex(unsigned long value) :_hierarchy_value(value),
5.         _previous_hierarchy_value(0) {}
6.     hierarchical_mutex(const hierarchical_mutex&) = delete;
7.     hierarchical_mutex& operator=(const hierarchical_mutex&) = delete;
8.     void lock() {
9.         check_for_hierarchy_violation();
10.        _internal_mutex.lock();
11.        update_hierarchy_value();
12.    }
13.
14.    void unlock() {
15.        if (_this_thread_hierarchy_value != _hierarchy_value) {
16.            throw std::logic_error("mutex hierarchy violated");
17.        }
18.
19.        _this_thread_hierarchy_value = _previous_hierarchy_value;
20.        _internal_mutex.unlock();
21.    }
22.
23.    bool try_lock() {
24.        check_for_hierarchy_violation();
25.        if (!_internal_mutex.try_lock()) {
26.            return false;
27.        }
28.
29.        update_hierarchy_value();
30.        return true;
31.    }
32. private:
33.    std::mutex  _internal_mutex;
34.    //当前层级值
35.    unsigned long const _hierarchy_value;
36.    //上一次层级值
37.    unsigned long _previous_hierarchy_value;
38.    //本线程记录的层级值
39.    static thread_local  unsigned long  _this_thread_hierarchy_value;
40.
41.    void check_for_hierarchy_violation() {
42.        if (_this_thread_hierarchy_value <= _hierarchy_value) {
```

```
43.             throw  std::logic_error("mutex  hierarchy violated");
44.         }
45.     }
46.
47.     void  update_hierarchy_value() {
48.         _previous_hierarchy_value = _this_thread_hierarchy_value;
49.         _this_thread_hierarchy_value = _hierarchy_value;
50.     }
51. };
52.
53. thread_local unsigned long hierarchical_mutex::_this_thread_hierarchy_value(ULONG_MAX);
54.
55. void test_hierarchy_lock() {
56.     hierarchical_mutex  hmtx1(1000);
57.     hierarchical_mutex  hmtx2(500);
58.     std::thread t1([&hmtx1, &hmtx2]() {
59.         hmtx1.lock();
60.         hmtx2.lock();
61.         hmtx2.unlock();
62.         hmtx1.unlock();
63.         });
64.
65.     std::thread t2([&hmtx1, &hmtx2]() {
66.         hmtx2.lock();
67.         hmtx1.lock();
68.         hmtx1.unlock();
69.         hmtx2.unlock();
70.         });
71.
72.     t1.join();
73.     t2.join();
74. }
```

层级锁能保证我们每个线程加锁时，一定是先加权重高的锁。
并且释放时也保证了顺序。
主要原理就是将当前锁的权重保存在线程变量中，这样该线程再次加锁时判断线程变量的权重和锁的权重是否大于，如果满足条件则继续加锁。

总结

本文介绍了线程互斥的常见问题和基本处理方案，在实际开发中，根据具体情境具体分析。

视频链接https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3435) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(539) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

喜欢(587) 浏览(1595)

解密定时器的实现细节

喜欢(566) 浏览(1449)

Linux环境搭建和编码

喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)